Assignment-5

UCS420 – "Cognitive Computing"
NumPy – Introduction-1 (Solved)

Q1. Create a NumPy 1-D array with 5 elements and perform basic operations.

```
import numpy as np

arr1 = np.array([1, 2, 3, 4, 5])
print("Original array:", arr1)
Original array: [1 2 3 4 5]
```

a) Addition of 2 to all elements.

```
print(arr1 + 2)
[3 4 5 6 7]
```

b) Multiply 3 with all elements.

```
print(arr1 * 3)
[ 3  6  9 12 15]
```

c) Divide every element by 2.

```
print(arr1 / 2)
[0.5 1.  1.5 2.  2.5]
```

Q2. Questions on Basic NumPy Array.

a) Reverse the NumPy array.

```
arr = np.array([1, 2, 3, 6, 4, 5])
reversed_arr = arr[::-1]
print(reversed_arr)
[5 4 6 3 2 1]
```

b) i. Find the most frequent value and its indices - x = [1,2,3,4,5,1,2,1,1,1]

```
x = np.array([1, 2, 3, 4, 5, 1, 2, 1, 1, 1])
vals, counts = np.unique(x, return_counts=True)
most_freq = vals[np.argmax(counts)]
indices = np.where(x == most_freq)[0]
print("Most frequent value =", most_freq)
print("Indices =", indices)
Most frequent value = 1
Indices = [0 5 7 8 9]
```

b) ii. Find the most frequent value and its indices - y = [1,1,1,2,3,4,2,4,3,3]

```
y = np.array([1, 1, 1, 2, 3, 4, 2, 4, 3, 3])
vals, counts = np.unique(y, return_counts=True)
most_freq = vals[np.argmax(counts)]
indices = np.where(y == most_freq)[0]
print("Most frequent value =", most_freq)
print("Indices =", indices)
Most frequent value = 1
Indices = [0 1 2]
```

Q3. For the given 2-D array, access elements using row and column indices.

```
arr2d = np.array([[10, 20, 30],
                  [40, 50, 60],
                  [70, 80, 90]])
print(arr2d)
[[10 20 30]
 [40 50 60]
 [70 80 90]]
```

a) Access 1st row, 2nd column.

```
print(arr2d[0, 1])
20
```

b) Access 3rd row, 1st column.

```
print(arr2d[2, 0])
70
```

Q4. Create a 1-D array named Gourav with 25 evenly spaced numbers from 10 to 100 using linspace(). Find dimensions, shape, size, dtype, nbytes, and transpose.

```
Gourav = np.linspace(10, 100, 25)
print(Gourav)

print("Dimensions:", Gourav.ndim)
print("Shape:", Gourav.shape)
print("Total elements:", Gourav.size)
print("Data type:", Gourav.dtype)
print("Total bytes:", Gourav.nbytes)
```

Output:

```
[ 10.    13.75  17.5   21.25  25.    28.75  32.5   36.25  40.    43.75
  47.5   51.25  55.    58.75  62.5   66.25  70.    73.75  77.5   81.25
  85.    88.75  92.5   96.25 100.  ]

Dimensions:   1
Shape:        (25,)
Total elements: 25
Data type:    float64
Total bytes:  200
```

Transpose using reshape():

```
Gourav_reshaped = Gourav.reshape(25, 1)
print(Gourav_reshaped.shape)
(25, 1)   -> converts the 1-D array into a 25x1 column vector
```

Can we do the same with the .T attribute?

```
Gourav_T = Gourav.T
print(Gourav_T.shape)
(25,)   -> shape is UNCHANGED
Answer: NO, not really. A 1-D array has only one axis, so .T has nothing
to swap and simply returns the same array with shape (25,) unchanged.
reshape(25, 1) actually creates a true 2-D column vector of shape (25, 1).
So to get a genuine "transpose-like" column vector from a 1-D array, we
must use reshape() - the .T attribute alone does not achieve this.
```

Q5. Create a 2-D array ucs420_Gourav (3x4), compute statistics, reshape and resize.

```
ucs420_Gourav = np.array([10, 20, 30, 40, 50, 60,
                          70, 80, 90, 15, 20, 35]).reshape(3, 4)
print(ucs420_Gourav)
[[10 20 30 40]
 [50 60 70 80]
 [90 15 20 35]]
```

Statistics:

```
print("Mean:", ucs420_Gourav.mean())
print("Median:", np.median(ucs420_Gourav))
print("Max:", ucs420_Gourav.max())
print("Min:", ucs420_Gourav.min())
print("Unique elements:", np.unique(ucs420_Gourav))
Mean:   43.333333333333336
Median: 37.5
Max:    90
Min:    10
Unique elements: [10 15 20 30 35 40 50 60 70 80 90]
```

Reshape to 4 rows x 3 columns:

```
reshaped_ucs420_Gourav = ucs420_Gourav.reshape(4, 3)
print(reshaped_ucs420_Gourav)
[[10 20 30]
 [40 50 60]
 [70 80 90]
 [15 20 35]]
```

Resize to 2 rows x 3 columns:

```
resized_ucs420_Gourav = np.resize(ucs420_Gourav, (2, 3))
print(resized_ucs420_Gourav)
[[10 20 30]
 [40 50 60]]
(np.resize takes the first 6 elements from the flattened array of 12
 elements to fit the new 2x3 = 6-element shape)
```